1. SLOWLY CHANGING DIMENSIONS (SCD)

SCDs are techniques to handle changes in dimension attributes over time while preserving the level of history required by the business for analysis.

SCDs apply to
- Dimension attributes (not fact measures).

Type	How it works	When to use	Example	Pros	Cons
Type 0 (Retain Original)	Keep original value, no changes	Value should never change	Date of birth, original joining date	Simple, maintains original value	Cannot correct errors
Type 1 (Overwrite)	Overwrite old value with new value	History not required (e.g. data correction)	Customer name spelling correction	Simple, less storage	No history lost information
Type 2 (New Row)	Insert new row with surrogate key, effective dates, current flag	Full history required	Customer changes city	Maintains complete history	More storage complexity
Type 3 (Additional Column)	Add columns to store current & previous value	Limited history required	Current and previous region	Less storage simple	Only limited history (not scalable)
Type 4 (Mini Dimension)	Move frequently changing attributes to separate mini-dim	Large dimension with rapidly changing attributes	Customer risk profile changes daily	Keeps main dimension stable, reduces volume	Additional join, extra table
Type 6 (1+2+3)	Combination of Type 1, 2 and 3	Need both historical and current view	Sales by historical and current customer segment	Supports multiple use cases	More complex implementation

Example: SCD Type 2 (Customer City Change)

Before Change

customer_sk	customer_id	city	effective_from	effective_to	is_current
501	C101	Delhi	2024-01-01	9999-12-31	Y

→

After Change (City → Bengaluru)

customer_sk	customer_id	city	effective_from	effective_to	is_current
501	C101	Delhi	2024-01-01	2025-01-01	N
502	C101	Bengaluru	2025-01-01	9999-12-31	Y

Key Implementation Steps for SCD Type 2

1. Identify business key (e.g. customer_id)
2. Detect changes in tracked attributes (null-safe comparison)
3. Expire old record (set effective_to, is_current = N)
4. Insert new record with new surrogate key and effective_from, is_current = Y
5. Resolve fact foreign keys based on event time
6. Validate (no overlapping dates, single current record, idempotent loads)

2. TYPES OF FACT TABLES

- Fact tables store measurable, quantitative data (facts) at a defined grain. Each row represents a business event, state or process depending on the type.

Type	What it represents	Grain / One row	Example	When to use
Transactional Fact	Individual business event	One row per transaction (e.g. order line)	Sales order line	Analyze customer behaviour, trends
Periodic Snapshot	State at regular time intervals	One row per entity per period (e.g. per day)	Daily inventory balance	Track activity over time, trend analysis
Accumulating Snapshot	Process with multiple milestones	One row per process instance (updated over time)	Order fulfillment (placed→shipped → delivered)	Track lifecycle and measure durations
Factless Fact	Event/relationship without measures	One row per event or relationship	Student attendance, website click	When only occurrence matters

Examples for Each Type

Transactional Fact – Sales Order Line

order_line_id	product	quantity	amount
OL101	Laptop	1	50000
OL102	Mouse	2	2000

One row per transaction (order line).

Periodic Snapshot – Inventory

date	product	warehouse	closing_stock
2024-10-10	Laptop	WH1	100
2024-10-11	Laptop	WH1	95

One row per product per day.
(Semi-additive measure across time)

Accumulating Snapshot – Order Fulfillment

order_id	placed_at	packed_at	shipped_at	delivered_at
0101	2024-10-10	2024-10-11	2024-10-12	NULL

One row per order (updated as milestones occur).

Factless Fact – Student Attendance

student_id	date	class_id
S101	2024-10-10	CSE101
S101	2024-10-11	CSE101

One row per event, no measures.

Types of Measures in Fact Tables

Measure type	Explanation	Example	Additive across
Additive	Can be summed across all dimensions	Sales revenue	Time, product, customer
Semi-additive	Can be summed across some dimensions (not time)	Inventory balance	Product, location
Non-additive	Cannot be meaningfully summed	Profit margin %	None (use avg, ratio)

3. TYPES OF DIMENSION TABLES

- Dimension tables store descriptive attributes that provide context to facts and help create meaningful value for analysis.

Type	Meaning	Example	Use case
SCD Dimension	Handles changing attributes over time (Type 0/1/2/3/4/6)	Customer, Product	Manage history for changing attributes
Conformed Dimension	Shared, consistent definition across multiple facts	Product, Customer	Common reporting across facts (e.g. Sales & Returns)
Degenerate Dimension	Transaction identifier stored in fact table (no separate dim table)	Order ID, Invoice No.	Useful for filtering and tracing transactions
Junk Dimension	Combines multiple low-cardinality flags	Promotion flag, Payment method, Gift wrap flag	Reduce joins for multiple indicators
Role-playing Dimension	Same dimension used in different roles	Date (Order Date, Ship Date, Delivery Date)	Multiple dates in a fact table
Static Dimension	Reference values that rarely or never change	Transaction type, currency, country code	Stable data, minimal maintenance
Shrunken Dimension	Smaller, higher-level subset of a dimension	Product Category (from Product)	Used for aggregate fact tables / higher level analysis

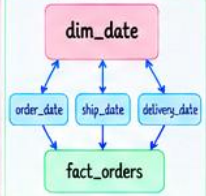
Examples

Conformed Dimension



Same product definition used across multiple fact tables.

Role-playing Dimension



Same Date dimension used in different roles (aliases).

Degenerate Dimension



Order ID stored in fact table for filtering and traceability.

4. SCENARIO BASED INTERVIEW QUESTIONS (WITH DETAILED ANSWERS)

1 Customer City Change (SCD Type 2)

Scenario: A customer has changed their city 10 times. The business needs every historical city and historical revenue by city. Which SCD type will you use? How will you implement it?

Answer:
- Use SCD Type 2.
- Maintain historical dimension versions with surrogate keys, effective_from, effective_to and current flag.
- Facts reference the customer_sk valid at transaction time.
- Validate no overlapping date ranges and only one current record per customer.

2 Rapidly Changing Risk Score (SCD Type 4)

Scenario: Customer risk score changes daily and your SOM row customer dimension is growing rapidly. What would you do?

Answer:
- Use SCD Type 4 (mini-dimension).
- Move frequently changing risk attributes to a separate mini-dimension (e.g. customer_risk profile).
- Fact table references both customer_sk and risk_profile_sk.
- Keeps main customer dimension stable and reduces volume.
- Evaluate cardinality and whether daily history is required.

3 Order Processing Time (Accumulating)

Scenario: Business needs order processing times across placed, packed, shipped and delivered stages. Which fact type will you use and how will you design it?

Answer:
- Use an accumulating snapshot fact table.
- Grain: one row per order.
- Store milestone timestamps (placed_at, packed_at, shipped_at, delivered_at).
- Update the same row as process progresses.
- Compute durations between milestones.
- Validate timestamp ordering and handle late arriving updates.

4 Sales & Returns Reporting (Conformed)

Scenario: Sales and Returns teams both need revenue and return reporting by product category. How should the dimension tables be designed?

Answer:
- Use a conformed Product dimension with consistent product IDs, categories and definitions across Sales and Returns.
- Keep measures in separate fact tables (fact_sales, fact_returns) at their correct grains.
- Aggregate each fact before combining them using common dimensions.
- Ensures consistent reporting and avoids duplicate or mismatched product definitions.

5 Multiple Dates in a Fact (Role-playing)

Scenario: Your order fact has order_date, ship_date and delivery_date. Do you create three separate Date dimension tables?

Answer:
- Usually no. Reuse the same Date dimension as a role-playing dimension.
- Join it multiple times with different aliases (order_date_key, ship_date_key, delivery_date_key).
- Maintains consistent date definitions and reduces maintenance.

6 Fact Table Design for Attendance (Factless)

Scenario: A school needs to track student attendance (no numeric measures). Which fact type will you use and why?

Answer:
- Use a factless fact table.
- Each row represents a student attendance event (student_id, date, class_id).
- No numeric measures, only the occurrence matters.
- Useful for reporting attendance, absence rates and engagement.

2. SCD TYPE 2 IMPLEMENTATION, ADVANCED CONCEPTS & BEST PRACTICES

1. SCD TYPE 2 IMPLEMENTATION - OVERVIEW & FLOW

Goal: Maintain historical records for changing dimension attributes using surrogate keys, effective dates and current flag. Each run should be idempotent (no duplicate versions) and handle late arriving data.

1. Identify business key (e.g. customer_id) → 2. Detect changes (null-safe comparison) → 3. Expire old record (set effective_to, is_current = N) → 4. Insert new record (new surrogate key, effective_from, is_current = Y) → 5. Validate (no overlaps, single current record)

2. SOURCE VS TARGET EXAMPLE

Source Data (stg_customer)

customer_id	name	city	updated_at
C101	Rahul	Delhi	2024-01-01
C101	Rahul	Bengaluru	2025-01-01
C102	Priya	Mumbai	2024-03-10

Target Dimension (dim_customer - Type 2)

customer_sk	customer_id	name	city	effective_from	effective_to	is_current
501	C101	Rahul	Delhi	2024-01-01	2025-01-01	N
502	C101	Rahul	Bengaluru	2025-01-01	9999-12-31	Y
601	C102	Priya	Mumbai	2024-03-10	9999-12-31	Y

3. SQL IMPLEMENTATION (MERGE - TYPE 2)

```
MERGE INTO dim_customer AS tgt
 USING stg_customer AS src
   ON tgt.customer_id = src.customer_id
     AND tgt.is_current = 'Y'
WHEN MATCHED AND (
      tgt.name <> src.name OR tgt.city <> src.city
    ) THEN
      UPDATE SET
         tgt.effective_to = DATE_SUB(src.updated_at, 1),
         tgt.is_current = 'N'

WHEN NOT MATCHED THEN
    INSERT (customer_sk, customer_id, name, city, effective_from, effective_to, is_current)
    VALUES (nextval('customer_sk_seq'), src.customer_id, src.name, src.city,
         src.updated_at, '9999-12-31', 'Y');
```

Key Points

- ✓ Compare only tracked attributes (null-safe).
- ✓ Use current record (is_current = Y) for change detection.
- ✓ Expire old record and insert new record in the same transaction.
- ✓ Use sequence/identity for surrogate key.
- ✓ Can be implemented using MERGE or separate UPDATE + INSERT.
- ✓ Use effective date from source (or load date if not available).

4. PYSPARK IMPLEMENTATION (SIMPLIFIED LOGIC)

```
from pyspark.sql.functions import col, when, lit, current_date
# 1. Read source and current target
  src = spark.table("stg_customer")
  tgt = spark.table("dim_customer").filter(col("is_current") == "Y")
# 2. Find changed and new records (null-safe comparison)
  changes = src.alias('s').join(
     tgt.alias('t'), "customer_id", "left")
  ).where(
     (tgt.customer_id.isNull()) |
     (col("s.name") != col("t.name"))| (col("s.city") != col("t.city"))
# 3. Prepare new records with new surrogate key
  from pyspark.sql.functions import monotonically_increasing_id
  new_records = changes.select(
     (monotonically_increasing_id() + 1000).alias("customer_sk"),
     col("s.customer_id"), col("s.name"), col("s.city"),
     col("s.updated_at").alias("effective_from"),
     lit("9999-12-31").alias("effective_to"), lit("Y").alias("is_current")
# 4. Expire old records (set effective_to, is_current = N)
  updated_tgt = tgt.alias('t').join(
     changes.select("customer_id","updated_at").alias("c"),
     col("t.customer_id") == col("c.customer_id")
).withColumn("effective_to", col("s.updated_at") - expr("INTERVAL 1 DAY"))
```

Notes

- ✓ Use window functions or MERGE if available (e.g. Databricks, Synapse).
- ✓ Ensure only one current record per business key.
- ✓ Use consistent effective date logic (load date vs source updated_at).
- ✓ Handle null-safe comparison.
- ✓ Do not remove old historical records, only expire current one.

5. IDEMPOTENCY IN SCD PIPELINES

Running the same input multiple times should not create duplicate or incorrect records.

How to make it idempotent?

- ✓ Use detreministic logic based on business key + tracked attributes.
- ✓ Compare using null-safe conditions.
- ✓ Only insert when a change is detected.
- ✓ Use MERGE/UPSERT instead of plain INSERT.
- ✓ Use consistent effective dates (e.g. source updated_at).
- ✓ Enforce unique constraint (customer_id, is_current = Y).
- ✓ Run post-load data quality checks.

Common Pitfalls (Causes Duplicates)

- ✗ Not handling null comparisons correctly.
- ✗ Different load timestamps causing multiple versions.
- ✗ Re-running INSERT without checking existing current record.
- ✗ No unique constraint on business key + current flag.
- ✗ Partial failures leaving expired record without new record.

6. DATA QUALITY VALIDATIONS (POST LOAD)

Check	Description	Example Query / Logic
Single current record	Each business key should have only one is_current = Y record	SELECT customer_id, COUNT(*) FROM dim_customer WHERE is_current = 'Y' GROUP BY customer_id HAVING COUNT(*) > 1
No overlapping dates	Effective date ranges should not overlap	Check using window functions: prev_effective_to < effective_from
No duplicate versions	Same attribute values should not create new row	Compare source vs latest record
No gaps (optional)	Continuous date ranges (if required)	Validate sequence of effective dates
Surrogate key unique	No duplicate customer_sk	Primary key / count distinct
Change detection	Only create new version when tracked attributes actually changed	Compare source vs current values

7. HANDLING LATE ARRIVING DATA

Late arriving dimension change: a change with effective date earlier than current latest record.

Approach:

1. Find the correct position in history based on effective_from.
2. Insert the new record with correct effective dates.
3. Adjust effective_to of previous record.
4. Ensure no overlapping date ranges.
5. Recalculate current flag if needed.

Example

Customer city updated on 2024-06-01 arrives late.

Insert the record with effective_from = 2024-06-01 and adjust neighbouring records' effective_to.

8. FAILURE HANDLING & RECOVERY

- Use atomic transactions (if supported).
- If using staging tables: write to staging → validate → swap/merge.
- On pipeline failure, re-run should be safe (idempotent).
- Use checkpoints and retry logic.
- Ensure no business key is left without a current record.

Post Failure Checks

- ✓ All open records have is_current = Y.
- ✓ No partial updates (expired without new record).
- ✓ Counts match expectations.
- ✓ Data quality validations pass.

9. SCD TYPE 4 - MINI DIMENSION (Detailed)

Used when some attributes change very frequently and storing full history in main dimension (Type 2) will cause large size. Create a separate mini-dimension for volatile attributes (e.g. customer risk profile).

dim_customer (stable attributes)
- customer_sk
- customer_id
- name, city, ...

→ **dim_customer_profile** (mini dimension)
- profile_sk
- risk_category
- risk_score
- segment, ...

→ **fact_sales**
- customer_sk
- profile_sk
- product_sk
- sales_amount

Example: Customer Risk Profile

dim_customer (Type 2 for stable attributes)

customer_sk	customer_id	name	city
501	C101	Rahul	Delhi
502	C101	Rahul	Bengaluru

dim_customer_profile (Type 2)

profile_sk	customer_id	risk_score	effective_from	is_current
1001	C101	650	2024-01-01	N
1002	C101	720	2024-07-01	Y

Key Points

- ✓ Fact table references both customer_sk and profile_sk.
- ✓ Keeps main customer dimension smaller and stable.
- ✓ Useful for frequently changing attributes.

10. SCD TYPE 6 - COMBINATION OF 1, 2 & 3 (Example)

Maintains historical rows like Type 2 and also keeps current attribute values like Type 3. Useful when both historical and current view are required together.

customer_sk	customer_id	hist_city	current_city	effective_from	effective_to	is_current
501	C101	Delhi	Bengaluru	2024-01-01	2025-01-01	N
502	C101	Bengaluru	Bengaluru	2025-01-01	9999-12-31	Y

Key Points

- ✓ Contains historical and current attribute values.
- ✓ Helpful for reporting: sales by historical city and current city.
- ✓ More storage than Type 3 but easier reporting.

11. DIFFERENT SCD POLICIES FOR DIFFERENT ATTRIBUTES

A dimension can have multiple attributes with different SCD strategies.

Attribute	SCD Type	Example	Reason
customer_id	Type 0	Business key	Never changes
date_of_birth	Type 0	DOB	Should not change
name	Type 1	Spelling correction	History not required
city	Type 2	Customer moves city	Full history required
email	Type 1	Correct data	History not required
risk_score	Type 4	Frequently changes	Use mini-dimension
segment	Type 3	Current and previous	Limited history required

12. ADVANCED DESIGN DECISIONS & TRADE-OFFS

Decision Area	Considerations
Storage vs History	Type 1 (less storage) vs Type 2 (more history) vs Type 4 (mini-dimension)
Query Performance	Star schema (denormalized) vs Snowflake (normalized)
Grain Selection	Match business process and reporting needs
Late Arriving Data	Use effective dates and reprocess logic
Fact Table Impact	Changing dimensions may require reprocessing facts or using current view (Type 6) for easier reporting
Data Lineage	Maintain audit columns (created_at, updated_at, batch_id)